COMPANION TO THE RAC ON KUBEADM LAB GUIDE

The Services, and why RAC needs them

Eight Kubernetes Services stand in for the host names, VIPs, SCAN addresses and network access a RAC cluster normally gets from DNS and the data-centre network. What each one is for, where Grid and clients use it, and how to see it working.

`Headless Services` `CoreDNS` `publishNotReadyAddresses` `NodePort` `SCAN · VIP · local_listener`

25 Oct 2026 · The lab follows Oracle's How to Install and Configure Oracle RAC on Oracle Cloud Native Environment. This page explains guide steps 10.3 and 13.1 to 13.4 from the network's point of view.

`PowerShell`

on your PC

`worker · controlplane`

root shell on a VM

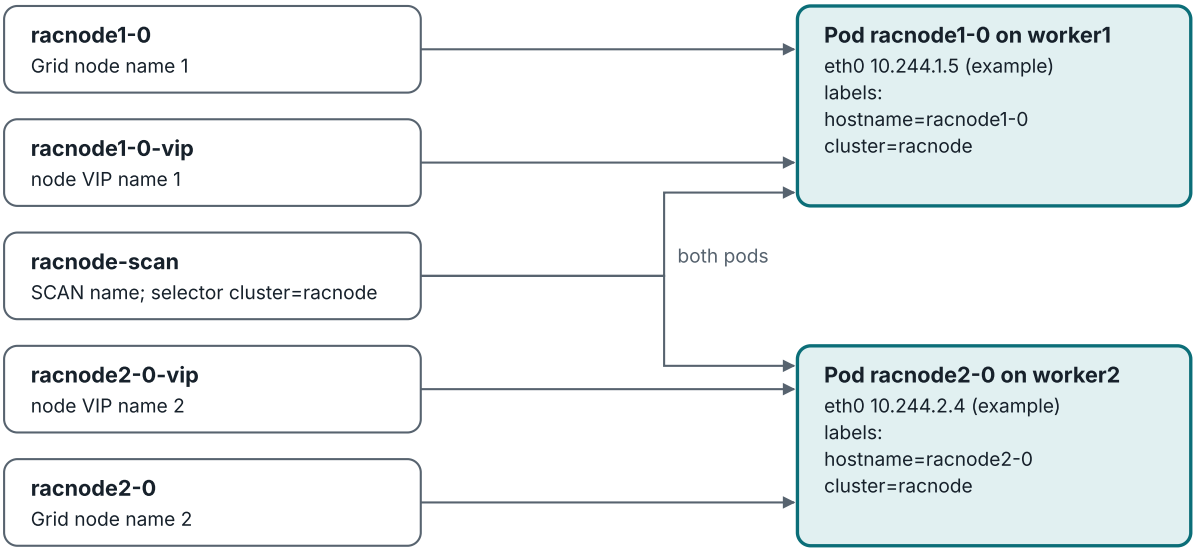
`racnode1-0`

root shell in a RAC pod

At a glance

The RAC pods need eight Kubernetes Services. Five are **headless** Services from guide 10.3: they exist only to put names in the cluster's DNS, the names Grid Infrastructure insists on. Three are **NodePort** Services from guide 13.3: they open ports on the nodes so clients outside the cluster can reach the listeners.

Five headless Services give Grid five names for two pods



CoreDNS answers racnode1-0.rac.svc.cluster.local, racnode1-0-vip..., racnode-scan... with these pod IPs.
Inside the pods the short names work too: the search domain rac.svc.cluster.local is added for you.

Each name resolves to its pod's own IP; racnode-scan returns both. Example IPs; they change when a pod is recreated.

SERVICE	TYPE	SELECTS	A LOOKUP RETURNS	USED BY	GUIDE
`racnode1-0`	headless	pod `racnode1-0`	racnode1-0's pod IP	Grid: node name 1, SSH between nodes, cluster checks	10.3
`racnode2-0`	headless	pod `racnode2-0`	racnode2-0's pod IP	Grid: node name 2	10.3
`racnode1-0-vip`	headless	pod `racnode1-0`	racnode1-0's pod IP	Grid: node VIP 1; `local_listener` after 13.3	10.3
`racnode2-0-vip`	headless	pod `racnode2-0`	racnode2-0's pod IP	Grid: node VIP 2; `local_listener` after 13.3	10.3
`racnode-scan`	headless	both pods (`cluster=racnode`)	**both** pod IPs	Grid: SCAN name; every client connection	10.3
`racnode-scan-lsnr`	NodePort 30200	both pods	(a port, not a name)	clients outside the cluster: first hop to a SCAN listener	13.3

SERVICE	TYPE	SELECTS	A LOOKUP RETURNS	USED BY	GUIDE
`racnode1-np`	NodePort 30203, 30201	pod `racnode1-0`	(ports)	outside clients redirected to instance 1; FAN (ONS)	13.3
`racnode2-np`	NodePort 30204, 30202	pod `racnode2-0`	(ports)	outside clients redirected to instance 2; FAN (ONS)	13.3

In one sentence each: the node names identify the two cluster members, the VIP names are the per-node addresses clients are sent to, the SCAN name is the single name clients connect to, and the NodePorts carry those same conversations in from outside Kubernetes.

Kubernetes background in five points

1. A Service selects pods by label. Every Service has a `selector`. Kubernetes keeps a list of the pods that match it, called its **endpoints** (stored as EndpointSlices). Our RAC pods carry two labels each, from the StatefulSets in guide 10.4:

POD	HOSTNAME LABEL	CLUSTER LABEL
`racnode1-0`	`racnode1-0`	`racnode`
`racnode2-0`	`racnode2-0`	`racnode`

The node and VIP Services select on the pod's name (`statefulset.kubernetes.io/pod-name`, which Kubernetes adds to every StatefulSet pod), so each matches exactly one pod. `racnode-scan` selects on `cluster: racnode`, so it matches both.

2. CoreDNS turns Services into names. Every Service gets the name `<service>.<namespace>.svc.cluster.local`. For our Services in namespace `rac`, that's for example `racnode-scan.rac.svc.cluster.local`. The `cluster.local` part comes from `dnsDomain` in the kubeadm configuration (guide 5.1).

3. The type decides what that name returns.

TYPE	CLUSTERIP	WHAT A LOOKUP RETURNS	HOW TRAFFIC FLOWS
ClusterIP (the default)	a virtual IP, such as `10.96.14.7`	that one virtual IP	kube-proxy rewrites it to one of the pods, chosen per connection
Headless	`None`	**the pod IPs themselves**, one A record per pod	straight to the pod; no translation
NodePort	a virtual IP, plus a port on every node	the virtual IP	any node's IP on the node port reaches a pod

The five RAC name Services are headless (`clusterIP: None`). Why that matters for Grid is covered in "Why headless and not a normal ClusterIP Service".

4. Short names work inside the pods. A pod's `/etc/resolv.conf` has the search list `rac.svc.cluster.local svc.cluster.local cluster.local` . So inside a RAC pod, `racnode2-0` is enough; the resolver tries `racnode2-0.rac.svc.cluster.local` for you. A pod in another namespace (such as the test client `dbclient` in `rac-client`) must use the full name.

5. Readiness normally hides pods from DNS. By default a Service only lists pods that are **Ready**. The RAC pods' readiness probe only passes once something answers on port 1521, which is after Grid is installed. `publishNotReadyAddresses: true` switches that off for these Services; its own section explains why it's essential.

Why RAC needs these names at all

Before Grid Infrastructure is installed on any cluster, bare metal or not, the network team has to provide a set of names and addresses. The installer checks every one of them resolves before it does anything. On bare metal they come from DNS or `/etc/hosts` ; here they come from the headless Services.

GRID NEEDS	ON BARE METAL	IN THIS LAB	WHERE GRID IS TOLD ABOUT IT
A public host name per node	DNS / `/etc/hosts` : `node1 → 192.168.1.11`	Service `racnode1-0` → racnode1-0's pod IP	`clusterNodes=racnode1-0:racnode1-0-vip:HUB,...` in the response file (11.3)
A virtual IP (VIP) per node	a spare address on the public subnet, `node1-vip → 192.168.1.21`	Service `racnode1-0-vip` → **the same** pod IP	the second field of each `clusterNodes` entry
A SCAN name with 1–3 addresses	`cluster-scan → 192.168.1.31, .32, .33` (round robin)	Service `racnode-scan` → both pod IPs	`scanName=racnode-scan` , `scanPort=1521`
A public network	the server LAN, for example `192.168.1.0/24`	the Flannel pod network `10.244.0.0/16` on `eth0`	`networkInterfaceList=eth0:10.244.0.0:1,...`
Private networks	dedicated NICs, no DNS needed	Multus `eth1` / `eth2` , fixed IPs (guide 6)	`networkInterfaceList=...,eth1:10.0.11.0:5,eth2:10.0.12.0:5`

Every address in the first three rows must also be **on the public network** Grid was given. The pod IPs are on `10.244.0.0/16` , which is exactly the public network in the response file. That's one reason the names must return pod IPs and nothing else (see "Why headless").

The private networks need no Services at all: the interconnect uses fixed IP addresses on `eth1` and `eth2` , which Grid discovers from the interfaces.

The node names: racnode1-0 and racnode2-0

What they are: the names of the two cluster members. They match the pods' host names, which come from the StatefulSet: pod `racnode1-0` has host name `racnode1-0`.

Where they're used:

- **The Grid installer** checks that each node name resolves and is reachable before installing (the cluster verification checks).
- **SSH between the nodes.** The installers on racnode1-0 copy the software to racnode2-0 and run commands there over SSH as `grid` and `oracle` (guide 11.1 sets up the keys and the `known_hosts` entries for these names).
- **Clusterware itself.** `olsnodes` lists the members by these names, and many resources are named after them (`ora.racnode1-0.vip`, for example).

Example, from inside a pod:

```
racnode1-0
CONNECT FIRST ssh -p 2210 root@127.0.0.1 , then kubectl -n rac exec -it racnode1-0 -- bash

hostname
getent hosts racnode1-0 racnode2-0
su - grid -c "olsnodes -n -s"
```

`hostname` prints `racnode1-0`; `getent` prints one pod IP per name, each with its full name `racnode1-0.rac.svc.cluster.local`; after Grid is installed `olsnodes` lists both nodes as `Active`.

Why a Service is needed even though the pod already knows its own name: a pod knows its own host name, but other pods can only find it through DNS, and DNS only has names for Services. Without these two Services, racnode1-0 couldn't resolve `racnode2-0`, and the installer would stop at its first check.

The VIP names: racnode1-0-vip and racnode2-0-vip

What a VIP is on bare metal: each RAC node has an extra public address, its virtual IP. Clients are sent to the VIP, not to the host address. If a node dies, Grid moves its VIP to a surviving node, where it immediately refuses connections. A client trying the dead node gets a quick "connection refused" instead of waiting for a TCP timeout, and moves on to the next address.

What it is here: `racnode1-0-vip` resolves to **racnode1-0's own pod IP**, the same address as `racnode1-0`. It's the doc's design, for a practical reason: an address can't move between workers. Flannel gives each worker its own slice of the pod network (for example `10.244.1.0/24` for worker1) and routes it only to that worker, so even if Grid moved the address to racnode2-0 on worker2, no packet for it would ever arrive there.

What still works, and what doesn't:

- Grid still has a VIP resource per node (`ora.racnode1-0.vip`), and its configuration is complete.
- **Fast failover through VIP relocation doesn't happen here.** Clients survive a node loss through **SCAN** instead: the SCAN name returns both pods, and the SCAN listeners only redirect to instances that are running.

- After guide 13.3, the VIP names gain a second job: each instance's `local_listener` names its VIP and a node port, for example `racnode1-0-vip.rac.svc.cluster.local:30203`. That's the address the SCAN listener tells clients to reconnect to.

`racnode1-0`

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash`

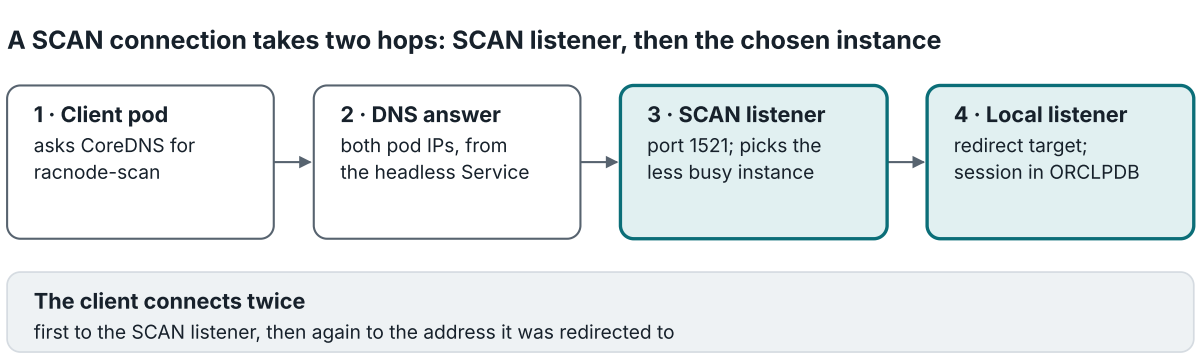
```
getent hosts racnode1-0 racnode1-0-vip
su - grid -c "srvctl config vip -n racnode1-0"
su - oracle -c "export ORACLE_SID=ORCLCDB1; echo 'show parameter local_listener' | sqlplus -s / as sysdba"
```

Both names print the **same** IP. `srvctl` shows the VIP's name and address. `local_listener` shows the pod IP on 1522 before guide 13.3, and the VIP name on 30203 after it.

The SCAN name: racnode-scan

What SCAN is: the Single Client Access Name. Clients connect to one name for the whole cluster and never need to know how many nodes there are. Grid starts a SCAN listener for each address the SCAN name returns.

Here: `racnode-scan` selects **both** pods (`cluster: racnode`), so CoreDNS returns two A records, one per pod. Grid creates two SCAN VIPs and two SCAN listeners: `LISTENER_SCAN1` and `LISTENER_SCAN2`, normally one in each pod. On bare metal you'd usually have three SCAN addresses; two is what two pods can give.



Inside the cluster. Before guide 13.3 the redirect target is the pod IP on port 1522; after it, the VIP name on 30203 or 30204.

A connection inside the cluster takes two hops:

1. The client asks CoreDNS for `racnode-scan.rac.svc.cluster.local` and gets both pod IPs.
2. It connects to port 1521 on one of them: a SCAN listener.
3. The SCAN listener knows which instances offer the requested service (`ORCLPDBSVC`) and how busy each is. It replies with a **redirect** to the chosen instance's local listener, at the address in that instance's `local_listener`.
4. The client opens a second connection to that address and gets its session.

The redirect is why the names matter so much: whatever address the instance registered, the client must be able to reach it. Inside the cluster both the pod IP and the VIP name work. Outside the cluster neither does on its own, which is what the NodePort Services and the Windows hosts entries are for.

controlplane

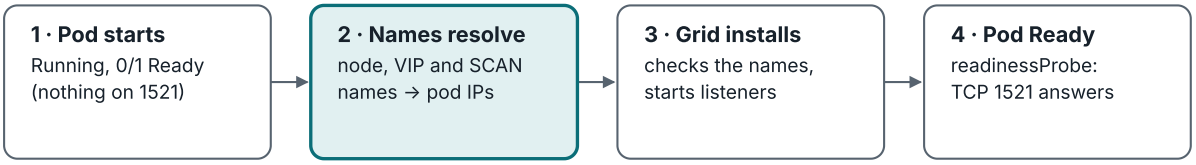
CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
kubectl -n rac get endpointslices -l kubernetes.io/service-name=racnode-scan
kubectl -n rac exec racnode1-0 -- getent ahostsv4 racnode-scan
kubectl -n rac exec racnode1-0 -- su - grid -c "srvctl config scan; srvctl status scan_listener"
```

The EndpointSlice and `getent` list both pod IPs. `srvctl` shows two SCAN VIPs and two SCAN listeners and the node each one runs on (normally one per node).

publishNotReadyAddresses: the chicken-and-egg it solves

The pod only becomes Ready after Grid runs, so DNS must answer before Ready



Without publishNotReadyAddresses
step 2 returns no address, Grid's checks fail at step 3, and step 4 never comes

publishNotReadyAddresses: true lets the headless Services publish the pod IPs from step 1.

The problem: Kubernetes normally publishes a pod in a Service's DNS only when the pod is **Ready**. The RAC pods are Ready only when port 1521 answers (the `readinessProbe` in guide 10.4), and nothing listens on 1521 until Grid is installed and its listeners start. But the Grid installer needs the node, VIP and SCAN names to resolve **before** it installs anything. Without help, each waits for the other forever.

The fix: `publishNotReadyAddresses: true` on all five headless Services. They publish the pod IPs as soon as the pods have them, Ready or not. This is also why `kubectl -n rac get pods` shows `0/1` until Grid is running, and that's expected.

Why keep the readiness probe at all: it still means something useful once Grid is up: `1/1` tells you the node's listener answers. Tools that look at readiness, such as `kubectl wait`, then reflect whether the node can serve connections.

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
kubectl -n rac get svc -o custom-columns=NAME:.metadata.name,TYPE:.spec.type,CLUSTER-
IP:.spec.clusterIP,NOTREADY:.spec.publishNotReadyAddresses
```

The five name Services show `CLUSTER-IP` `None` and `NOTREADY` `true`.

Why headless and not a normal ClusterIP Service

A normal Service would give each name one stable virtual IP, which sounds attractive for a database. For Grid it breaks three things:

1. **The address must be on Grid's public network.** Grid's public network is the pod network `10.244.0.0/16`. A ClusterIP comes from the separate Service range (`10.96.0.0/12` by default) and isn't on any interface, so Grid's checks that each node, VIP and SCAN address belongs to the public network would fail.
2. **Grid must reach the exact pod.** A virtual IP is load-balanced by kube-proxy per connection. `racnode-scan` as a ClusterIP would hide the two SCAN listeners behind one address, and Grid couldn't bring up or check one SCAN VIP per node.
3. **The redirect must reach the instance the listener chose.** With headless Services the address in the redirect is the instance's own pod (or its VIP name, which resolves to that pod). Nothing in between can send the second connection to the wrong pod.

What headless gives up: the name's address changes when a pod is recreated and gets a new IP. Grid copes because it stores the node, VIP and SCAN entries as **names** and resolves them again when it starts (the restart test in guide 14.4 shows it).

The NodePort Services: reaching RAC from outside the cluster

A client outside Kubernetes, such as Python on your Windows PC, has two problems: it can't resolve `*.svc.cluster.local` names, and it can't route to pod IPs. A **NodePort** Service opens one port on **every** node and forwards it to the selected pods; kube-proxy delivers it whichever node it arrives on.

Oracle's document adds three NodePort Services (guide 13.3):

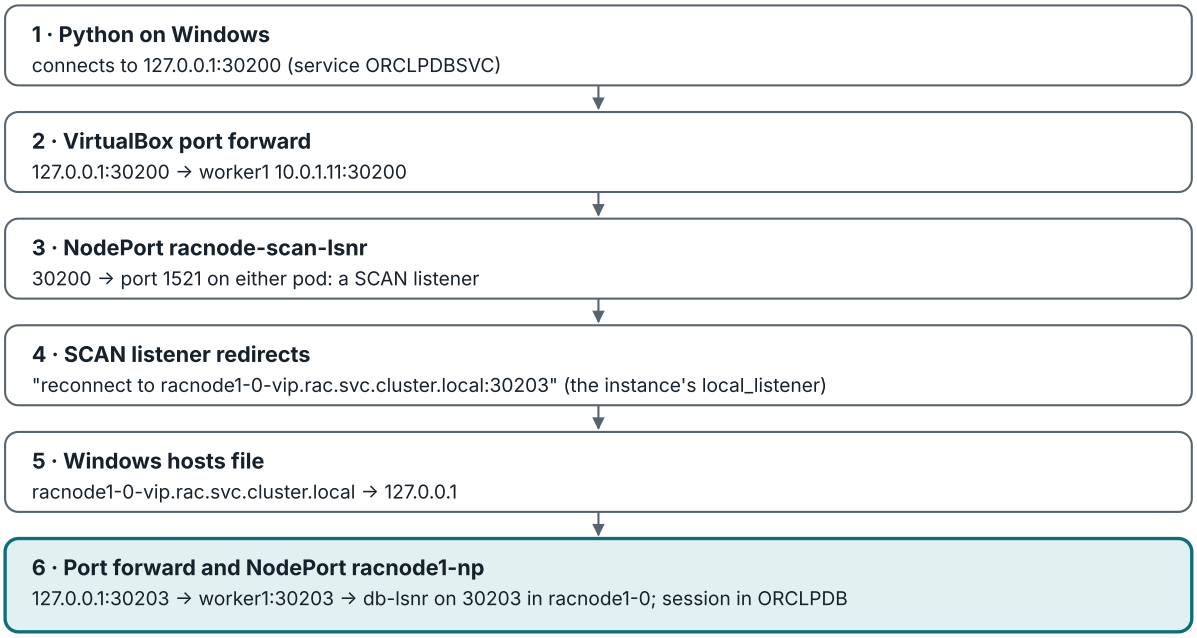
SERVICE	NODE PORT → POD PORT	SELECTS	FOR
`racnode-scan-lsnr`	30200 → 1521	both pods	the first hop: any SCAN listener
`racnode1-np`	30203 → 30203, 30201 → 6200	racnode1-0	the redirect to instance 1's listener `db-lsnr`; ONS (FAN events)

SERVICE	NODE PORT → POD PORT	SELECTS	FOR
`racnode2-np`	30204 → 30204, 30202 → 6200	racnode2-0	the redirect to instance 2; ONS

Why the node port and the pod port are the same number for the redirect (30203 → 30203): the SCAN listener's redirect names a port, and the client uses that port exactly as given. The instance can only advertise one port, and it must work both inside the pod and on the node. So the doc adds a second node listener, `db-lsnr`, listening **in the pod** on 30203 and 30204, and points each instance's `local_listener` at its VIP name and that port.

Why the redirect names the VIP and not an IP: a name can mean different things in different places. Inside the cluster CoreDNS resolves `racnode1-0-vip.rac.svc.cluster.local` to the pod. On Windows the hosts file resolves it to `127.0.0.1`, where the VirtualBox port forward takes over. The same redirect works for both kinds of client.

From Windows, a connection crosses two NodePorts and one SCAN redirect



The lab path from guide 13.4. Steps 2 and 6 use VirtualBox port forwards to worker1; any node would do, kube-proxy finds the pod.

ONS ports (30201, 30202): Oracle Notification Service carries FAN events, for example "instance 2 is down", so connection pools can drop dead sessions at once. The lab exposes them as the doc does but doesn't test FAN from Windows.

`controlplane`

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
kubectl -n rac get svc racnode-scan-lsnr racnode1-np racnode2-np
kubectl -n rac exec racnode1-0 -- su - grid -c "srvctl config listener -listener db-lsnr"
```

The three Services show their node ports; `db-lsnr` listens on `30203,30204`.

Try it yourself

Every Service, its selector and type:

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
kubectl -n rac get svc -o 'custom-columns=NAME:.metadata.name,TYPE:.spec.type,CLUSTER-IP:.spec.clusterIP,SELECTOR:.spec.selector,NODEPORTS:.spec.ports[*].nodePort'
kubectl -n rac get endpointslices -o 'custom-columns=SERVICE:.metadata.labels.kubernetes\.io/service-name,ADDRESSES:.endpoints[*].addresses'
kubectl -n rac get pods -o wide
```

The pod IPs in `get pods` are exactly the addresses in the EndpointSlices of the five headless Services; `racnode-scan` has both.

What a pod sees, short and full names:

racnode1-0

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash`

```
cat /etc/resolv.conf
for h in racnode1-0 racnode2-0 racnode1-0-vip racnode2-0-vip racnode-scan; do echo "$h -> $(getent ahostsv4 $h | awk '{print $1}' | sort -u | xargs)"; done
```

What a client in another namespace sees (it must use full names):

controlplane

CONNECT FIRST `ssh -p 2210 root@127.0.0.1`

```
kubectl -n rac-client exec dbclient -- getent ahostsv4 racnode-scan.rac.svc.cluster.local
kubectl -n rac-client exec dbclient -- getent ahostsv4 racnode-scan || echo "short name doesn't resolve from another namespace, as expected"
```

Where the SCAN listener sends clients (each instance's registered address):

racnode1-0

CONNECT FIRST `ssh -p 2210 root@127.0.0.1` , then `kubectl -n rac exec -it racnode1-0 -- bash`

```
su - grid -c "lsnrctl services LISTENER_SCAN1" | grep -iE 'service "orclpdbsvc|instance|REMOTE SERVER|ADDRESS'
```

The handlers listed under `ORCLPDBSVC` show the addresses the SCAN listener redirects to: pod IPs on 1522 before guide 13.3, the VIP names on 30203 and 30204 after it. `LISTENER_SCAN1` may run in either pod; if `lsnrctl` says it isn't running there, run the same in racnode2-0, or use `LISTENER_SCAN2`.

What breaks without each Service

MISSING OR WRONG	SYMPTOM	WHEN YOU'D SEE IT
`racnode1-0` / `racnode2-0`	the other node's name doesn't resolve; SSH between nodes and the installer's checks fail	Grid install (guide 11.4)
`racnode1-0-vip` / `racnode2-0-vip`	the installer reports the VIP names don't resolve	Grid install
`racnode-scan`	SCAN name doesn't resolve: install fails; later, clients can't connect at all	Grid install, every client
`racnode-scan` with a selector matching one pod	only one SCAN listener; clients still connect, but lose SCAN if that pod is down	node failure
`publishNotReadyAddresses` missing	names resolve to nothing while the pods are `0/1`, so the install can't start	Grid install
A ClusterIP instead of headless	addresses not on Grid's public network; checks fail	Grid install
`racnode-scan-lsnr` (NodePort)	outside clients get "connection refused" on 30200	Windows test (13.5)
`racnode1-np` / `racnode2-np`	the first hop works, the redirected second connection fails	Windows test
hosts entries on Windows	the redirect name doesn't resolve on Windows: "could not resolve"	Windows test
port forwards 30200/30203/30204	"actively refused" on 127.0.0.1 (issue #36 in the first build)	Windows test

In production

- **Applications inside the cluster** are the simplest case: they use `racnode-scan.rac.svc.cluster.local:1521` and the headless Services exactly as in this lab, with no NodePorts at all.
- **Clients outside the cluster** need a stable way in. NodePorts plus hosts-file entries work for a lab; in production you'd use DNS records in your corporate DNS for the VIP and SCAN names, pointing at the nodes (or at a load balancer) that carry the node ports, so every client resolves them without local changes. A `LoadBalancer` Service can replace the first hop, but the redirect still needs each instance's own reachable name and port, so it doesn't remove the per-instance Services.

- **VIP failover** doesn't happen with an overlay pod network like Flannel, because an address can't move between workers. Plan on SCAN plus FAN (through ONS) for fast client failover, and test it.
- **GNS** (Grid Naming Service, where Grid manages its own DNS sub-domain) isn't used: CoreDNS and the headless Services take its place.
- **Keep the names stable.** Grid stores names, not IPs. Renaming a Service, the namespace or the cluster domain after the install means reconfiguring Grid.

Built from the lab's build log and scripts (rac-k8s-poc). Values are from the author's lab; change them to match yours.